Problem: A Men at work

Every morning you have to drive to your workplace. Unfortunately, roads are under constant repair. Fortunately, administration is aware that this may cause trouble and they enforce a strict rule on roadblocks: roads must be blocked only half of the time. However, contractors are free to schedule their working hours, still they must follow regulations:

- Working periods (when the road is blocked) and rest periods (when the road is open) must alternate and be of fixed length.
- The beginning of the day (time zero) must coincide with the beginning of a period.

1 Problem

Write a program that, given a description of the road network and of contractors schedules outputs the minimal time needed to drive from home to work.

2 Input specification

The road network is represented on a $N \times N$ grid and the first line of the input consists in the number N, $2 \leq N \leq 50$.

Then, come N lines of N characters that represent the road network at time zero. Those lines are made of ``.'' (standing for open road) and ``*'' (standing for roadblock) and they encode the rows of the grid in increasing order, while columns are also presented in increasing order. Conventionally, your home is at the position first row, first column, while your workplace is at the position last row, last column. Furthermore, you leave home at time t=0, that is, your starting position is first row, first column at time zero.

At a given time t, your car must be on some ``open road'' cell. It takes one unit of time to drive to any of the four adjacent cells heading toward north, south, west or east, and you may also choose to stay on the same cell for one unit of time. Of course, those five moves are valid if and only if the targeted cell exists and is free at time t+1.

Finally, come N lines of N characters that represent the contractors schedules. Those lines match the ones of the grid description and are made of N characters `0`, `1`, ..., `9` that specify the duration of the working (and rest) period for a given cell. Observe that `0` is a bit special, since it means that the corresponding cell status does not change.

3 Output specification

The output consist in a single line, holding either the requested time, or `NO`, if driving from home to work is not possible.

4 Sample input

```
10
.*********
.........**
*.******.*
*.******.*
*.******.*
*........*
*.******.*
*.******.*
*........*
********..
0000000000
0000000000
0000000000
```

```
0000000000
0000000000
0123456780
0000000000
0000000000
0123456780
0000000000
```

5 Sample output

```
34
```

6 Sample input

```
3
...
**.
**.
021
002
000
```

7 Sample output

```
NO
```

This document was translated from L^AT_EX by H^EV^EA.

Problem B: Square dance

You are hired by the french NSA to break the RSA code used on the Pink Card. The easiest way to do that is to factor the public modulus and you have found the fastest algorithm to do that, except that you have to solve a subproblem that can be modeled in the following way.

Let $P = \{p_1, p_2, \ldots, p_n\}$ be a set of prime numbers. We call *relation* a set of two primes $\{p, q\}$, where p and q are distinct. You have a collection of R relations $S_i = \{p_i, q_i\}$. If S and T are sets of primes, then $S * T$ will denote the product of all the primes in S and T. You are interested in subproducts of the (S_i)'s whose product make a square.

The way you look for these squares is the following. The ultimate goal is to count squares that appear in the process. Relations arrive one at a time. You maintain a collection C of relations that do not contain any square subproduct. This is easy: at first, C is empty. Then a relation arrives and C begins to grow. Suppose a new relation $\{p, q\}$ arrives. If no square appears when adding $\{p, q\}$ to C, then $\{p, q\}$ is added to the collection. Otherwise, a square is about to appear, we increase the number of squares, **but we do not store this relation**, hence C keeps the desired property.

Let us consider an example. First arrives $S_1 = \{2, 3\}$ and we put it in C; then arrive $S_2 = \{5, 11\}$, $S_3 = \{3, 7\}$ and they are stored in C. Now enters the relation $S_4 = \{2, 7\}$. This relation could be used to form the square:

$$S_1 * S_3 * S_4 = (2 * 3) * (3 * 7) * (2 * 7) = (2 * 3 * 7)^2.$$

So we count 1 and do not store S_4 in C. Now, we consider $S_5 = \{5, 11\}$ that could make a square with S_2, so we count 1 square more. Then $S_6 = \{2, 13\}$ is put into C. Now $S_7 = \{7, 13\}$ could make the square $S_1 * S_3 * S_6 * S_7$. Eventually, we get 3 squares.

1 Input

Each test case begins with a line containing two integers P and R: $P \leq 10^6$ is the number of primes occurring in the test case; R ($\leq 10^6$) is the number of sets of primes that arrive. The subsequent R lines each contain two integers i and j making a set $\{p_i, p_j\}$ ($1 \leq i, j \leq P$). Note that we actually do not deal with the primes, that are irrelevant to the solution.

2 Output

For each test case, output the number of squares that can be formed using the preceding rules. A line contains the number of the test case, followed by the number of square.

3 Examples

For instance, the following file contains the first example and a second one:

```
6 7
1 2
3 5
2 4
1 4
3 5
1 6
4 6
```

The program must output:

```
3
```

For the second example:

```
2 3
1 2
1 2
1 2
```

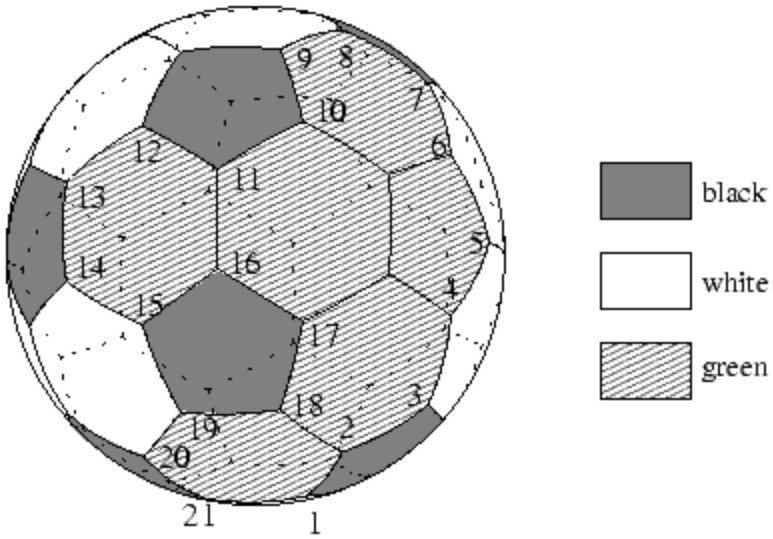
the output will be:

```
2
```

This document was translated from L^AT_EX by H^EV^EA.

Problem C: Football

Eric has a classic football that is made of 32 pieces of leather: 12 black pentagons and 20 white hexagons. Each pentagon adjoins 5 hexagons and each hexagon adjoins 3 pentagons and 3 hexagons. Eric drew a polygon (i.e. a closed line without intersections) along the edges of the pieces. The polygon divided the ball into two parts and Eric painted one of them green.



He is curious if given the description of the polygon you are able to compute the number of black, white and green pieces?

1 Task

Write a program that:

- reads the description of the polygon from the standard input,
- computes the number of black, white and green pieces,
- writes the result to the standard output.

2 Input

The first line of the input contains one integer n being the number of vertices of the polygon. The second line of the input contains n integers $a_1, a_2, ..., a_n$ separated by single spaces. Integer a_i (equal 1 or 2) is the number of green pieces adjoining the i-th vertex of the polygon. The side of the polygon connecting the n-th and the first vertex always lies between two hexagons.

3 Output

The first and only line of the output contains three integers b, w and g. These are the numbers of black, white and green pieces respectively.

4 Example

For the input:

```
21
1 2 1 2 1 2 1 1 1 2 2 1 1 1 1 2 2 2 1 1 1
```

the correct answer is:

```
11 15 6
```

This document was translated from L^AT_EX by H^EV^EA.

Problem D: Rent your airplane and make money

``ABEAS Corp.'' is a very small company that owns a single airplane. The customers of ABEAS Corp are large airline companies which rent the airplane to accommodate occasional overcapacity.

Custommers send renting orders that consist of a time interval and a price that the custommer is ready to pay for renting the airplane during the given time period.

Orders of all the custommers are known in advance. Of course, not all orders can be acomodated and some orders have to be declined. Eugene LAWLER, the Chief Scientific Officer of ABEAS Corp would like to maximize the profit of the company.

You are requested to compute an optimal solution.

1 Small Example

Consider for instance the case where the company has 4 orders AF514 (start time 0, duration 5, price 10), CO5 (start time 3, duration 7, price 14), AF515 (start time 5, duration 9, price 7) and BA01 (start time 6, duration 9, price 8). The optimal solution consists in declining CO5 and AF515 and the gain is 10 + 8 = 18. Note that the solution made of AF514 and AF515 is feasible (the airplane is rented with no interruption from time 0 to time 14) but non-optimal.

2 Input Data

Each test case is described by one `input` file that contains all the relevant data: The first line contains the number n of orders ($n \leq 3000$). This line is followed by n lines. Each of which describes an order and contains the name identifier of the order (less than 80 characters, with no white space) followed by three integer values: the start time of the order, the duration of the order and the price the custommer is ready to pay for this order (white spaces are used as separators). We assume that orders have a strictly positive duration.

3 Sample Input

```
4
AF514 0 5 10
CO5 3 7 14
AF515 5 9 7
BA01 6 9 8
```

4 Output

You are required to compute an optimal solution for each input file. Your program has to write the total price paid by airlines.

5 Sample Output

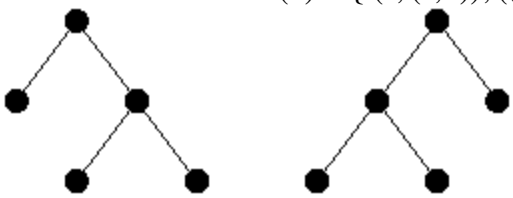
The output corresponding to the input file is:

```
18
```

This document was translated from L^AT_EX by H^EV^EA.

Problem: E Who's next?

Every computer science student knows binary trees. Here is one of many possible definitions of binary trees. Binary trees are defined inductively. A binary tree t is either an external node (leaf) • or a single ordered pair (t_1,t_2) of two binary trees, left subtree t_1 and right subtree t_2 respectively, called an internal node. Given an integer n, $B(n)$ is the set of trees with n leaves. For instance, the picture below shows the two trees of $B(3) = \{ (\bullet, (\bullet, \bullet)), ((\bullet, \bullet), \bullet) \}$.



Observe that those trees both have two internal nodes and a total of five nodes.

Given a tree t we define its unique integer identifier $N(t)$:

1. $N(\bullet) = 0$
2. $N(t_1, t_2) = 2^{n_1 + n_2} + 2^{n_2} N(t_1) + N(t_2)$, where n_1 and n_2 are the number of nodes in t_1 and t_2 respectively.

For instance, we have $N(\bullet, \bullet) = 2^2 + 2^1 \times 0 + 0 = 4$, $N(\bullet, (\bullet, \bullet)) = 2^4 + 2^3 \times 0 + 4 = 20$ and $N((\bullet, \bullet), \bullet) = 2^4 + 2^1 \times 4 + 0 = 24$.

The ordering $\succcurlyeq$ is defined on binary trees as follows:

$$\bullet \succcurlyeq t$$

$$(t_1, t_2) \succcurlyeq (u_1, u_2), \text{when } t_1 \prec u_1, \text{ or } t_1 = u_1 \text{ and } t_2 \succcurlyeq u_2$$

Hence for instance, $(\bullet, (\bullet, \bullet)) \succcurlyeq ((\bullet, \bullet), \bullet)$ holds, since we have $\bullet \prec (\bullet, \bullet)$.

Using the ordering $\succcurlyeq$, $B(n)$ can be sorted. Then, given a tree t in $B(n)$, we define $S(t)$ as the tree that immediately follows t in the sorted presentation of $B(n)$, or as the smallest tree in $B(n)$, if t is maximal in $B(n)$. For instance, we have $S(\bullet, \bullet) = (\bullet, \bullet)$ and $S(\bullet, (\bullet, \bullet)) = ((\bullet, \bullet), \bullet)$.

By composing the inverse of N, S and N we finally define a partial map on integers.

$$s(n) = N(S(N^{-1}(n)))$$

1 Task

Write a program that computes $s(n)$.

2 Input

The first input line contains an integer K, with $K > 0$. It is followed by K lines, each specifying an integer n_i with $1 \le i \le K$ and $0 \le n_i < 2^{31}$.

3 Output

The output should consist of K lines, the ith output line being $s(n_i)$, or NO if $s(n_i)$ does not exist.

4 Example

For the input:

```
5
4
0
20
5
432
```

The correct answer is:

```
4
0
24
NO
452
```

This document was translated from L^AT_EX by H^EV^EA.

Problem F: Computer

Little Tom is learning how to program. He has just written some programs but is afraid to run them, because he does not know if they will ever stop. Please write a program to help him.

This is not as easy a task as it may seem, because the computer Tom has is not deterministic.

Given a program written by Tom, your program should tell him whether his program can possibly stop and if so, what is the shortest possible time before it stops.

The computer consists of 32 1-bit registers and the program consists of n instructions. The registers are numbered from 0 to 31 and the instructions are numbered from 0 to n-1.

Below, `MEM[a]` stands for the contents of the a-th register, $0 \le a,b < 32$, $0 \le x < n$, $0 \le c \le 1$.

The instruction set is the following:

Instruction	Semantics
`AND a b`	`MEM[a]` := `MEM[a]` and `MEM[b]`
`OR a b`	`MEM[a]` := `MEM[a]` or `MEM[b]`
`XOR a b`	`MEM[a]` := `MEM[a]` xor `MEM[b]`
`NOT a`	`MEM[a]` := not `MEM[a]`
`MOV a b`	`MEM[a]` := `MEM[b]`
`SET a c`	`MEM[a]` := c
`RANDOM a`	`MEM[a]` := random value (0 or 1)
`JMP x`	jump to instruction number x
`JZ x a`	jump to instruction number x if `MEM[a]` = 0
`STOP`	stop the program

The last instruction of a program is always `STOP` (although there can be more than one `STOP` instruction). Every program starts with the instruction number 0. Before the start, the contents of the registers are random. Each instruction (including `STOP`) takes 1 processor cycle to execute.

1 Task

Write a program that:

- reads the program from the standard input,
- computes the shortest possible running time of the program,
- writes the result to the standard output.

2 Input

The first line of the input contains integer n ($1 \le n \le 16$) being the number of instructions of the program. Each of the next n lines contains one instruction of the program in the format given above.

3 Output

The first and only line of the output should contain the shortest possible running time of the program, measured in processor cycles. If the program cannot stop, output should contain the word HANGS.

4 Example

For the input:

```
5
SET 0 1
JZ 4 0
RANDOM 0
JMP 1
STOP
```

the correct answer is:

```
6
```

For the input:

```
5
MOV 3 5
NOT 3
AND 3 5
JZ 0 3
STOP
```

the correct answer is:

```
HANGS
```

This document was translated from $LaTeX$ by $HeVeA$.

Problem G: Help R2-D2!

In Episode III of Star Wars (whose alleged title is *How I became Vader*), R2-D2 (Artoo-Detoo) is again confronted to a tedious work. He is responsible for the loading of the republic transport starships in the fastest way. Imagine a huge space area where n starships are parked. Each starship has a capacity of K cubic femtoparsec. Containers C_i arrive one at a time with some volume v_i (expressed in cubic femtoparsec). R2-D2 wants to minimize the number of starships used for a given sequence of containers.

Smart as he is, R2-D2 knows for sure that the problem is a hard one, even with the force being around. Here is the heuristics he selected to solve his problem. Start with all starships ready to load, and numbered S_0, S_1, etc.. When container C_j arrives, select the starship of minimal index i that can contain C_j and put it in S_i. In some sense, this heuristics minimizes the move of the container arriving before its loading.

At the end of the n arrivals, R2-D2 counts the number s of starships used and he measures the *total waste* w of the sequence. For i=0..s-1, the *waste* in starship i is given by the unused volume.

Your task is to simulate the algorithm of R2-D2.

1 Input

Each test file begins with capacity K on a line ($K \leq 1000$), followed by the number of containers in the sequence, n on the second line ($1 \leq n \leq 10^6$). There are two possible formats for the remaining lines. If it contains one integer, then this is the next v_i. If it begins with the character `b` (for block), it is followed by 2 integers r and v. This means that the r next containers arriving have volume v.

2 Output

Your program must output the number s of starships used, followed by a blank, followed by the total waste w.

3 Examples

The first example file contains:

```
100
3
50
25
70
```

You load starship S_0 with 50 and 25 and starship S_1 with 70, so that the waste is (100-75)+(100-70)=55. The answer must be

```
2 55
```

Example 2 contains:

```
100
4
50
b 2 40
20
```

which corresponds to the sequence 50, 40, 40, 20. S_0 will contain 90, S_1 will contain 60, so that the waste is 10+40=50 and the answer will be:

This document was translated from L^AT_EX by H^EV^EA.

Problem H: Phony primes

You are chief debugger for Poorly Guarded Privacy, Inc. One of the top selling product, ReallySecureAgent©, seems to have a problem with its prime number generator. It produces from time to time bogus primes N. After a while, you realize that the problem is due to the way primes are recognized.

Every phony prime N you discover can be characterized as follows. It is odd and has distinct prime factors, say $N = p_1 * p_2 * \ldots * p_k$ with $p_i \neq p_j$, where the number k of factors is at least 3. Moreover, for all i=1..k, p_i-1 divides N-1. For instance, 561 = 3 * 11 * 17 is a phony prime.

Intrigued by this phenomenon, you decide to write a program that enumerates all such N's in a given interval $[N_{min}, N_{max}[$, with $1 \leq N_{min} < N_{max} < 2^{31}$, $N_{max} - N_{min} < 10^7$.

1 Input

Each test file contains one line. On this line are written the two integers N_{min} and N_{max} separated by a blank.

2 Output

For each test file, output the list of phony primes in increasing order, one per line. If there are no phony prime in the interval, then simply output `none` on a line.

3 Examples

If the file contains:

```
10 2000
```

the output will be:

```
561
1105
1729
```

and if the file contains:

```
20000 21000
```

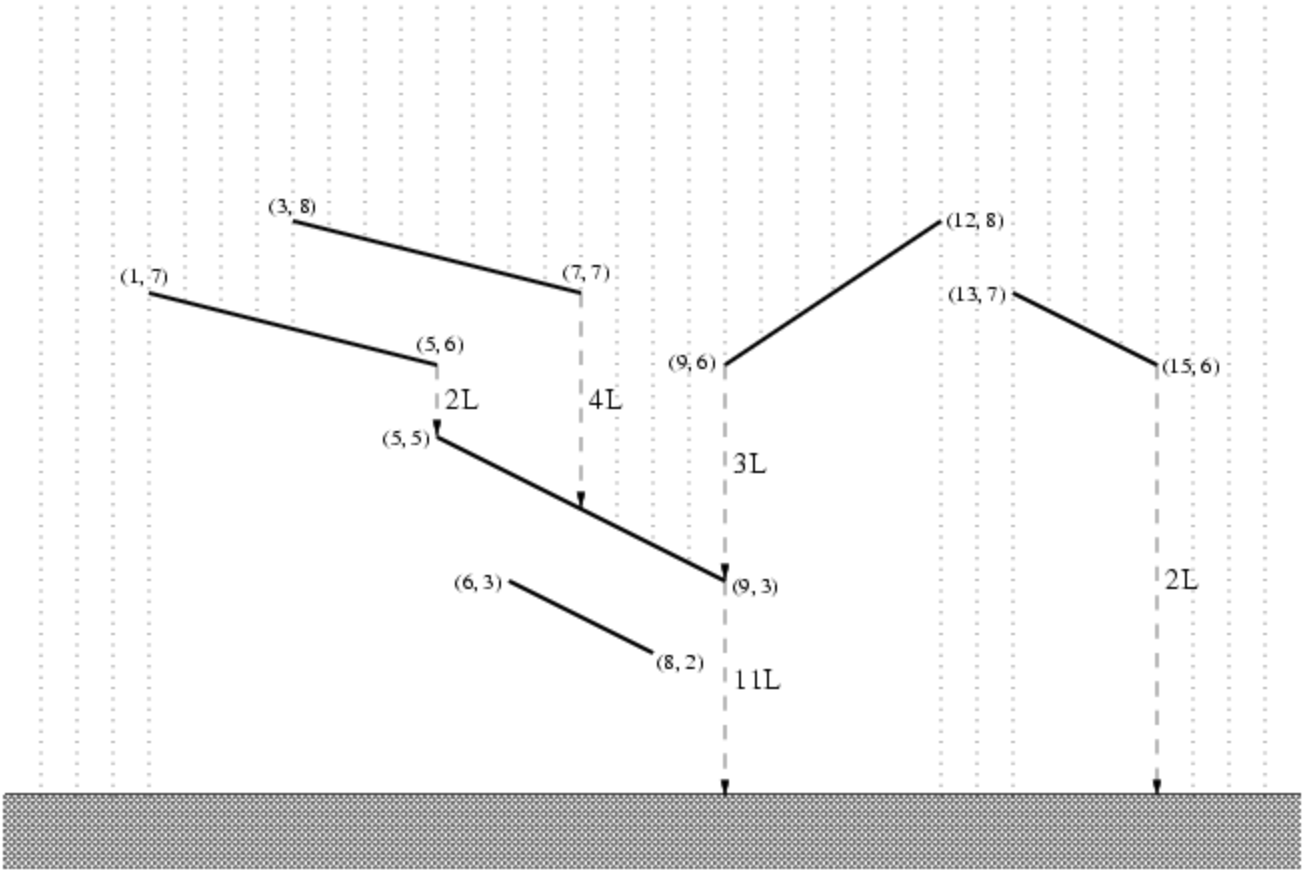
then the output will be

```
none
```

This document was translated from L^AT_EX by H^EV^EA.

Problem I: Roof

Contemporary buildings can have very complicated roofs. If we take a vertical section of such a roof it amounts to a number of sloping segments. When it is raining the drops are falling down onto the roof straight from the sky above. Some segments are completely exposed to the rain but there may be some segments partially or even completely shielded by other segments. All the water falling onto a segment flows as a stream straight down from the lower end of the segment on the ground or possibly onto some other segment. In particular if a stream of water is falling on an end of a segment then we consider it to be collected by this segment.



For the purpose of designing a piping system it is desired to compute how much water is flowing down from each segment of the roof. To be prepared for a heavy November rain you should count one liter of rain water falling on a meter of the horizontal plane during one second.

1 Task

Write a program that:

- reads the description of the roof from the standard input,
- computes the amount of water flowing down in one second from each segment of the roof,
- writes the result to the standard output.

2 Input

The first line of the input contains one integer n ($1 \le n \le 40\,000$) being the number of segments of the roof. Each of the next n lines describes one segment of the roof and contains four integers x_1, y_1, x_2, y_2 ($0 \le x_1, y_1, x_2, y_2 \le 1\,000\,000, x_1 < x_2, y_1 \neq y_2$) separated by single spaces. Integers x_1, y_1 are respectively the horizontal position and the height of the left end of the segment. Integers x_2, y_2 are respectively the horizontal position and the height of the right end of the segment. The segments don't have common points and there are no horizontal segments. Also you can assume that there are at most 100 segments placed above any point on the ground level.

3 Output

The output consists of n lines. The i-th line should contain the amount of water (in liters) flowing down from the i-th segment of the roof during one second.

4 Example

For the input:

```
6
13 7 15 6
3 8 7 7
1 7 5 6
5 5 9 3
6 3 8 2
9 6 12 8
```

the correct answer is:

```
2
4
2
11
0
3
```

This document was translated from L^AT_EX by H^EV^EA.